

**Facultad de Ciencias de la Computación**

Materia: Aplicaciones Distribuidas

# **Práctica Banco - Austro: Implementación de una Base de Datos Distribuida**

**Estudiante:** Isaias Abraham Urbina Romero

14 de julio de 2026

# Introducción

Las bases de datos distribuidas permiten almacenar información en diferentes servidores físicos o lógicos, ofreciendo mayor disponibilidad, escalabilidad y tolerancia a fallos. En este tipo de arquitectura, los datos se encuentran fragmentados o replicados entre varios nodos, mientras que el usuario interactúa con el sistema como si se tratara de una única base de datos.

En la presente práctica se desarrolló una aplicación distribuida para el Banco del Austro utilizando Spring Boot, PostgreSQL y Docker. Se implementaron dos nodos de base de datos correspondientes a las oficinas de Cuenca y Quito, cada uno con su propia información de clientes, cuentas y transacciones. La aplicación determina automáticamente a qué base de datos conectarse mediante el prefijo del número de cuenta, permitiendo realizar consultas de manera transparente para el usuario.

## 1 Objetivos

### Objetivo general

Desarrollar una aplicación distribuida utilizando Spring Boot y PostgreSQL que permita consultar información bancaria almacenada en dos nodos independientes, aplicando técnicas de fragmentación horizontal y enrutamiento de consultas.

### Objetivos específicos

- Configurar dos bases de datos PostgreSQL mediante contenedores Docker.
- Implementar múltiples conexiones (DataSource) dentro de Spring Boot.
- Enrutar automáticamente las consultas hacia el nodo correspondiente según el prefijo del número de cuenta.
- Realizar consultas distribuidas que permitan obtener información desde ambos nodos.
- Verificar el funcionamiento de la aplicación mediante pruebas de los servicios REST.

## 2 Desarrollo

### 2.1 Creación del proyecto

Para el desarrollo de la aplicación se creó un proyecto utilizando Spring Boot, administrado mediante Maven. La estructura del proyecto se organizó siguiendo la arquitectura propuesta por Spring, separando la configuración, la lógica del negocio y los controladores REST en paquetes independientes.

La aplicación fue desarrollada en Java y utiliza el framework Spring Boot para facilitar la creación de servicios web y la administración de múltiples conexiones hacia diferentes bases de datos PostgreSQL.

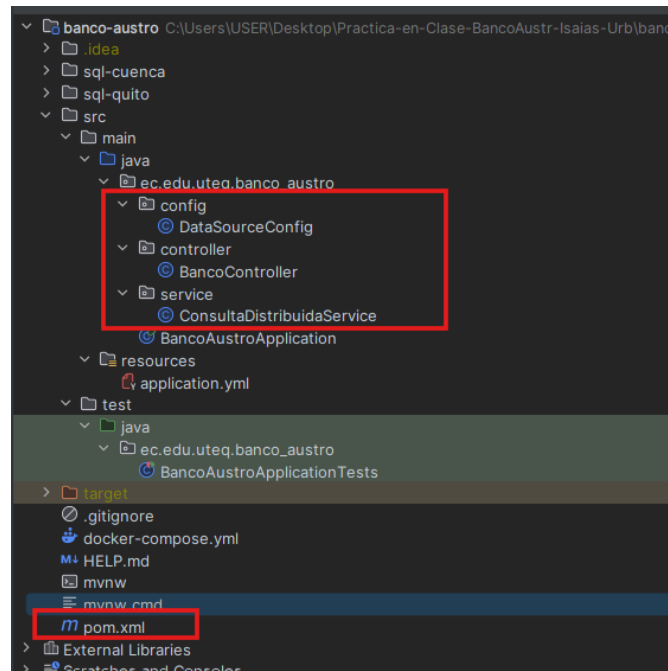


Figura 1: Estructura del Proyecto

## 2.2 Configuración de los nodos PostgreSQL mediante Docker

Con el propósito de simular una base de datos distribuida, se implementaron dos nodos PostgreSQL utilizando contenedores Docker. Cada nodo representa una oficina diferente del Banco del Austro:

- Nodo Cuenca, encargado de almacenar la información correspondiente a las cuentas cuyo prefijo es 22.
- Nodo Quito, encargado de almacenar la información correspondiente a las cuentas cuyo prefijo es 17.

Cada contenedor posee una base de datos independiente con sus propias tablas y registros iniciales. La creación automática de las estructuras y los datos se realizó mediante scripts SQL ejecutados durante el inicio de cada contenedor.

La configuración de ambos servicios se definió en el archivo `docker-compose.yml`, donde se especificaron las imágenes utilizadas, credenciales de acceso, puertos, volúmenes persistentes y las carpetas que contienen los scripts de inicialización (a continuación el código del `docker-compose.yml`):

Listing 1: docker-compose.yml

```

1 services:
2
3   db-cuenca:
4     image: postgres:16-alpine
5     container_name: banco-cuenca
6     environment:
7       POSTGRES_DB: banco_cuenca
8       POSTGRES_USER: banco
9       POSTGRES_PASSWORD: banco123
10    ports:
11      - "5442:5432"
12    volumes:
13      - vol-cuenca:/var/lib/postgresql/data

```

```

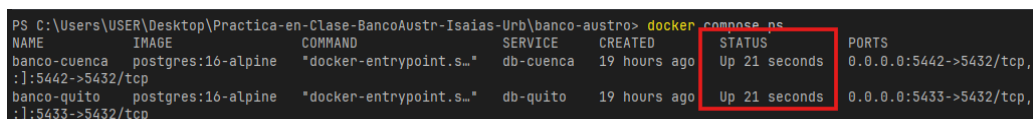
14     - ./sql-cuenca:/docker-entrypoint-initdb.d
15     networks:
16     - bda-net
17
18     db-quito:
19     image: postgres:16-alpine
20     container_name: banco-quito
21     environment:
22     POSTGRES_DB: banco_quito
23     POSTGRES_USER: banco
24     POSTGRES_PASSWORD: banco123
25     ports:
26     - "5433:5432"
27     volumes:
28     - vol-quito:/var/lib/postgresql/data
29     - ./sql-quito:/docker-entrypoint-initdb.d
30     networks:
31     - bda-net
32
33     networks:
34     bda-net:
35     driver: bridge
36
37     volumes:
38     vol-cuenca:
39     vol-quito:

```

Una vez configurados los contenedores, se inició el entorno utilizando Docker Compose. Al ejecutar el comando correspondiente se verificó que ambos nodos se encontraban activos y escuchando en diferentes puertos:

- Cuenca: puerto 5442 (ajustado para evitar conflicto con otra instalación de PostgreSQL).
- Quito: puerto 5433.

Esto permitió que la aplicación pudiera conectarse de forma independiente a cada base de datos.



| NAME         | IMAGE              | COMMAND                  | SERVICE   | CREATED      | STATUS        | PORTS                   |
|--------------|--------------------|--------------------------|-----------|--------------|---------------|-------------------------|
| banco-cuenca | postgres:16-alpine | "docker-entrypoint.s..." | db-cuenca | 19 hours ago | Up 21 seconds | 0.0.0.0:5442->5432/tcp, |
| banco-quito  | postgres:16-alpine | "docker-entrypoint.s..." | db-quito  | 19 hours ago | Up 21 seconds | 0.0.0.0:5433->5432/tcp, |

Figura 2: Resultado de `docker compose ps`

## 2.3 Creación de las bases de datos

Cada nodo contiene tres tablas principales:

- **clientes**, donde se almacena la información personal de cada cliente.
- **cuentas**, que registra el número de cuenta, el saldo disponible y la oficina a la que pertenece.
- **transacciones**, utilizada para almacenar el historial de movimientos realizados.

La estructura de las tablas es similar en ambos nodos; sin embargo, cada base de datos almacena únicamente la información correspondiente a su oficina. De esta manera se implementa una fragmentación horizontal por ubicación geográfica, donde cada servidor administra únicamente una parte de la información total del sistema.

Listing 2: 01\_schema.sql – Cuenca

```

1 CREATE TABLE IF NOT EXISTS clientes (
2   id BIGSERIAL PRIMARY KEY,
3   cedula VARCHAR(10) UNIQUE NOT NULL,
4   nombre VARCHAR(120) NOT NULL,
5   ciudad VARCHAR(60) NOT NULL,
6   creado_en TIMESTAMP NOT NULL DEFAULT NOW()
7 );
8
9 CREATE TABLE IF NOT EXISTS cuentas (
10  id BIGSERIAL PRIMARY KEY,
11  numero VARCHAR(20) UNIQUE NOT NULL,
12  cliente_id BIGINT NOT NULL REFERENCES clientes(id),
13  saldo NUMERIC(15,2) NOT NULL DEFAULT 0.00
14    CHECK (saldo >= 0),
15  oficina VARCHAR(20) NOT NULL CHECK (oficina = 'CUENCA'),
16  abierta_en TIMESTAMP NOT NULL DEFAULT NOW()
17 );
18
19 CREATE TABLE IF NOT EXISTS transacciones (
20  id BIGSERIAL PRIMARY KEY,
21  cuenta_orig VARCHAR(20) NOT NULL,
22  cuenta_dest VARCHAR(20) NOT NULL,
23  monto NUMERIC(15,2) NOT NULL CHECK (monto > 0),
24  fecha TIMESTAMP NOT NULL DEFAULT NOW(),
25  estado VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA',
26  oficina VARCHAR(20) NOT NULL CHECK (oficina = 'CUENCA')
27 );
28
29 CREATE INDEX idx_cuentas_cliente ON cuentas(cliente_id);
30 CREATE INDEX idx_transacciones_orig ON transacciones(cuenta_orig);
31 CREATE INDEX idx_transacciones_fecha ON transacciones(fecha);

```

Listing 3: 02\_datos.sql – Cuenca

```

1 INSERT INTO clientes (cedula, nombre, ciudad) VALUES
2 ('0101234567', 'Mario Vintimilla', 'Cuenca'),
3 ('0107654321', 'Sofia Vasquez', 'Cuenca'),
4 ('0103456789', 'Andres Cordero', 'Cuenca');
5
6 INSERT INTO cuentas (numero, cliente_id, saldo, oficina) VALUES
7 ('2201000001', 1, 15000.00, 'CUENCA'),
8 ('2201000002', 2, 4200.50, 'CUENCA'),
9 ('2201000003', 3, 850.75, 'CUENCA');
10
11 INSERT INTO transacciones (cuenta_orig, cuenta_dest, monto, oficina) VALUES
12 ('2201000001', '2201000002', 500.00, 'CUENCA'),
13 ('2201000002', '2201000003', 120.25, 'CUENCA');

```

## 2.4 Configuración de Spring Boot

### Configuración del archivo application.yml

Para permitir la conexión con las dos bases de datos distribuidas, se configuró el archivo `application.yml`. En este archivo se definieron dos orígenes de datos (DataSource): uno para la oficina de Cuenca y otro para la oficina de Quito.

Cada conexión contiene la dirección de la base de datos, el usuario, la contraseña y el controlador

JDBC de PostgreSQL. Debido a un conflicto con una instalación local de PostgreSQL, el nodo de Cuenca fue configurado para utilizar el puerto 5442, mientras que el nodo de Quito utiliza el puerto 5433.

Esta configuración permite que la aplicación pueda establecer conexiones independientes hacia cada nodo según sea necesario.

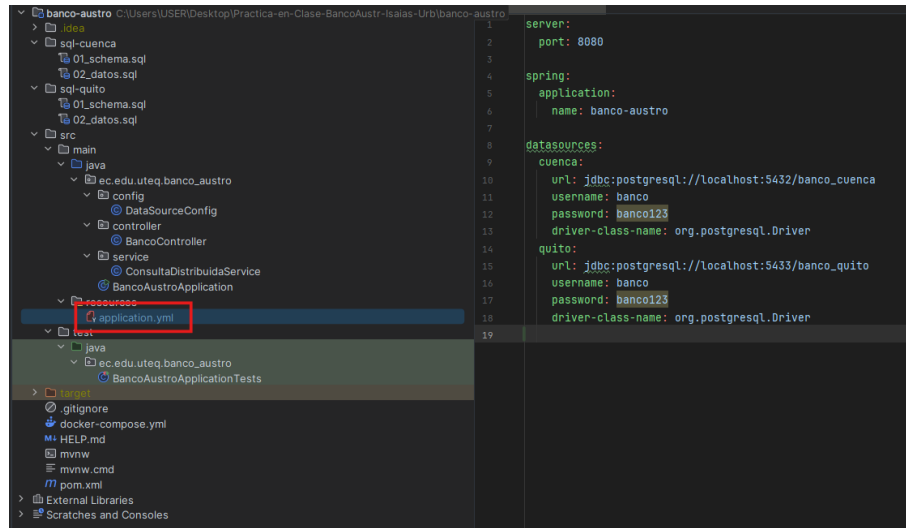


Figura 3: Captura del archivo application.yml

## 2.5 Configuración de los DataSource

Una vez definidas las conexiones en el archivo de configuración, se implementó la clase `DataSourceConfig`, encargada de crear los dos objetos `DataSource` que utilizará la aplicación.

Cada objeto representa una conexión independiente hacia una base de datos diferente y posteriormente es utilizado por el servicio para ejecutar las consultas correspondientes. Esta separación facilita la administración de múltiples bases de datos dentro de una misma aplicación Spring Boot.

Durante el inicio de la aplicación se imprimen en la consola las direcciones de conexión de ambos nodos, permitiendo verificar que la configuración fue cargada correctamente.

Listing 4: DataSourceConfig.java

```
1 package ec.edu.uteq.banco_austro.config;
2
3 import javax.sql.DataSource;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.boot.jdbc.DataSourceBuilder;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8
9 @Configuration
10 public class DataSourceConfig {
11
12     @Value("${datasources.cuenca.url}")
13     private String cuencaUrl;
14
15     @Value("${datasources.cuenca.username}")
16     private String cuencaUser;
17 }
```

```

18     @Value("${datasources.cuenca.password}")
19     private String cuencaPass;
20
21     @Value("${datasources.quito.url}")
22     private String quitoUrl;
23
24     @Value("${datasources.quito.username}")
25     private String quitoUser;
26
27     @Value("${datasources.quito.password}")
28     private String quitoPass;
29
30     @Bean(name = "dsCuenca")
31     public DataSource dsCuenca() {
32
33         System.out.println("===== CONFIGURACION CUENCA =====");
34         System.out.println("URL : " + cuencaUrl);
35         System.out.println("USER : " + cuencaUser);
36         System.out.println("PASS : " + cuencaPass);
37
38         return DataSourceBuilder.create()
39             .url(cuencaUrl)
40             .username(cuencaUser)
41             .password(cuencaPass)
42             .driverClassName("org.postgresql.Driver")
43             .build();
44     }
45
46     @Bean(name = "dsQuito")
47     public DataSource dsQuito() {
48
49         System.out.println("===== CONFIGURACION QUITO =====");
50         System.out.println("URL : " + quitoUrl);
51         System.out.println("USER : " + quitoUser);
52         System.out.println("PASS : " + quitoPass);
53
54         return DataSourceBuilder.create()
55             .url(quitoUrl)
56             .username(quitoUser)
57             .password(quitoPass)
58             .driverClassName("org.postgresql.Driver")
59             .build();
60     }
61 }

```

## 2.6 Implementación del servicio distribuido

La lógica principal del sistema fue implementada en la clase `ConsultaDistribuidaService`, responsable de determinar automáticamente a qué base de datos debe dirigirse cada consulta.

Para ello, la aplicación analiza los dos primeros dígitos del número de cuenta ingresado por el usuario.

- Si el número comienza con 22, la consulta es enviada al nodo correspondiente a Cuenca.
- Si el número comienza con 17, la consulta es enviada al nodo correspondiente a Quito.

Este mecanismo de enrutamiento permite que el usuario no necesite conocer la ubicación física

de la información, ya que la aplicación selecciona automáticamente la base de datos adecuada.

Además del enrutamiento por prefijo, el servicio implementa una consulta que obtiene la lista completa de clientes combinando la información almacenada en ambos nodos. Para ello, primero consulta la base de datos de Cuenca y posteriormente la de Quito, uniendo los resultados en una sola colección que es enviada al cliente.

Esta estrategia demuestra el funcionamiento de una consulta distribuida, donde la información se obtiene desde diferentes servidores y se presenta de forma unificada.

Listing 5: ConsultaDistribuidaService.java

```
1 package ec.edu.uteq.banco_austro.service;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.Map;
6 import javax.sql.DataSource;
7 import org.springframework.beans.factory.annotation.Qualifier;
8 import org.springframework.jdbc.core.JdbcTemplate;
9 import org.springframework.stereotype.Service;
10
11 @Service
12 public class ConsultaDistribuidaService {
13
14     private final JdbcTemplate jdbcCuenca;
15     private final JdbcTemplate jdbcQuito;
16
17     public ConsultaDistribuidaService(
18         @Qualifier("dsCuenca") DataSource dsCuenca,
19         @Qualifier("dsQuito") DataSource dsQuito) {
20         this.jdbcCuenca = new JdbcTemplate(dsCuenca);
21         this.jdbcQuito = new JdbcTemplate(dsQuito);
22     }
23
24     public Map<String, Object> consultarSaldo(String numero) {
25         JdbcTemplate destino = enrutar(numero);
26         String sql = "SELECT numero, saldo, oficina "
27             + "FROM cuentas WHERE numero = ?";
28         List<Map<String, Object>> filas =
29             destino.queryForList(sql, numero);
30         if (filas.isEmpty()) {
31             return Map.of(
32                 "error", "Cuenta no encontrada",
33                 "numero", numero);
34         }
35         return filas.get(0);
36     }
37
38     public List<Map<String, Object>> listarTodosLosClientes() {
39         String sql = "SELECT cedula, nombre, ciudad FROM clientes";
40         List<Map<String, Object>> union = new ArrayList<>();
41         union.addAll(jdbcCuenca.queryForList(sql));
42         union.addAll(jdbcQuito.queryForList(sql));
43         return union;
44     }
45
46     private JdbcTemplate enrutar(String numero) {
47         if (numero == null || numero.length() < 2) {
```



```

48         throw new IllegalArgumentException(
49             "Numero de cuenta invalido: " + numero);
50     }
51     String prefijo = numero.substring(0, 2);
52     return switch (prefijo) {
53         case "22" -> jdbcCuenca;
54         case "17" -> jdbcQuito;
55         default -> throw new IllegalArgumentException(
56             "Prefijo de oficina no reconocido: " + prefijo);
57     };
58 }
59 }

```

## 2.7 Implementación del controlador REST

Para exponer la funcionalidad de la aplicación se implementó la clase `BancoController`, la cual define los servicios REST disponibles para los usuarios.

Se desarrollaron dos endpoints principales:

- **Consultar saldo**, que recibe el número de cuenta y devuelve la información correspondiente al nodo donde se encuentra almacenada.
- **Listar clientes**, que obtiene los registros existentes en ambas bases de datos y los presenta como una única respuesta.

El controlador recibe las solicitudes HTTP, delega el procesamiento al servicio distribuido y devuelve la información en formato JSON.

Listing 6: BancoController.java

```

1 package ec.edu.uteq.banco_austro.controller;
2
3 import java.util.List;
4 import java.util.Map;
5 import ec.edu.uteq.banco_austro.service.ConsultaDistribuidaService;
6 import org.springframework.web.bind.annotation.GetMapping;
7 import org.springframework.web.bind.annotation.PathVariable;
8 import org.springframework.web.bind.annotation.RequestMapping;
9 import org.springframework.web.bind.annotation.RestController;
10
11 @RestController
12 @RequestMapping("/api/banco")
13 public class BancoController {
14
15     private final ConsultaDistribuidaService service;
16
17     public BancoController(ConsultaDistribuidaService service) {
18         this.service = service;
19     }
20
21     @GetMapping("/saldo/{numero}")
22     public Map<String, Object> saldo(@PathVariable String numero) {
23         return service.consultarSaldo(numero);
24     }
25
26     @GetMapping("/clientes")
27     public List<Map<String, Object>> clientes() {
28         return service.listarTodosLosClientes();
29     }
30 }

```

29  
30

```
}  
}
```

## 2.8 Ejecución de la aplicación

Una vez configurados los contenedores Docker y desarrollada la aplicación en Spring Boot, se procedió a ejecutar el proyecto desde el entorno de desarrollo IntelliJ IDEA.

Durante el inicio de la aplicación se verificó la correcta carga de los dos orígenes de datos configurados para las oficinas de Cuenca y Quito. Posteriormente, Spring Boot inicializó el servidor web embebido Tomcat en el puerto 8080, permitiendo el acceso a los servicios REST implementados.

Además, se comprobó que ambos grupos de conexiones (HikariPool) lograron establecer comunicación con las bases de datos correspondientes, confirmando que la configuración distribuida funcionó correctamente.

```
===== CONFIGURACIÓN CUENCA =====  
URL : jdbc:postgresql://localhost:5442/banco_cuenca  
USER : banco  
PASS : banco123  
===== CONFIGURACIÓN QUITO =====  
URL : jdbc:postgresql://localhost:5433/banco_quito  
USER : banco  
PASS : banco123  
2020-07-14T21:02:59.467-05:00 INFO 27928 --- [banco-austro] [      main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'  
2020-07-14T21:02:59.485-05:00 INFO 27928 --- [banco-austro] [      main] e.e.u.b.BancoAustroApplication : Started BancoAustroApplication in 2.354 seconds (process running for 2.769)  
2020-07-14T21:03:01.811-05:00 INFO 27928 --- [banco-austro] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'  
2020-07-14T21:03:01.812-05:00 INFO 27928 --- [banco-austro] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'  
2020-07-14T21:03:01.815-05:00 INFO 27928 --- [banco-austro] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Complete initialization in 1 ms  
2020-07-14T21:03:01.875-05:00 INFO 27928 --- [banco-austro] [nio-8080-exec-1] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting..  
2020-07-14T21:03:02.163-05:00 INFO 27928 --- [banco-austro] [nio-8080-exec-1] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@252c3753  
2020-07-14T21:03:02.164-05:00 INFO 27928 --- [banco-austro] [nio-8080-exec-1] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
```

Figura 4: Inicio exitoso de la aplicación Banco Austro, mostrando el arranque del servidor Tomcat en el puerto 8080 y la inicialización del pool de conexiones HikariCP hacia la base de datos PostgreSQL.

## 3 Pruebas realizadas

### 3.1 Consulta de una cuenta perteneciente a Cuenca

Se realizó una consulta utilizando el endpoint encargado de obtener la información de una cuenta bancaria. Se ingresó el número de cuenta 2201000001, cuyo prefijo corresponde a la oficina de Cuenca.

La aplicación identificó automáticamente el prefijo 22 y direccionó la consulta hacia la base de datos del nodo Cuenca. Como resultado, se obtuvo correctamente el número de cuenta, el saldo disponible y la oficina a la que pertenece.

Endpoint utilizado: GET /api/banco/saldo/2201000001

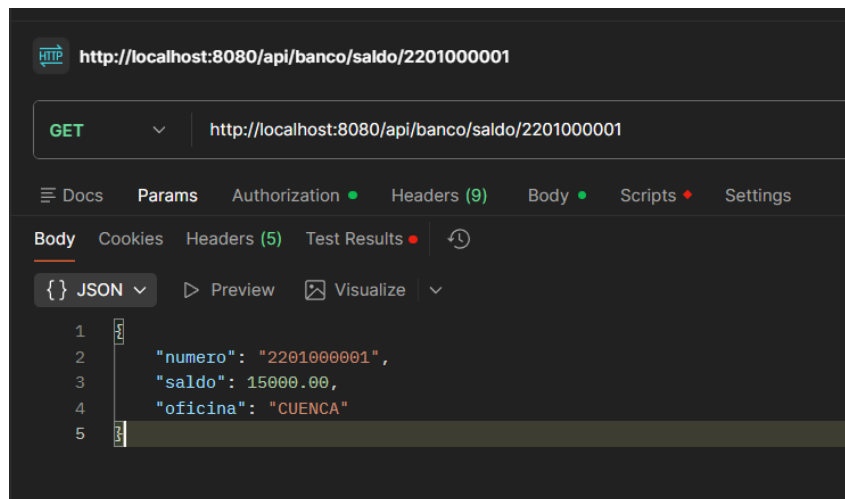


Figura 5: Captura de Postman mostrando la respuesta JSON.

### 3.2 Consulta de una cuenta perteneciente a Quito

Posteriormente se consultó una cuenta registrada en el nodo Quito mediante el número 1701000001.

En este caso, la aplicación detectó el prefijo 17 y dirigió automáticamente la consulta hacia la segunda base de datos. La información obtenida correspondió correctamente a la cuenta almacenada en dicho nodo.

Endpoint utilizado: GET /api/banco/saldo/1701000001

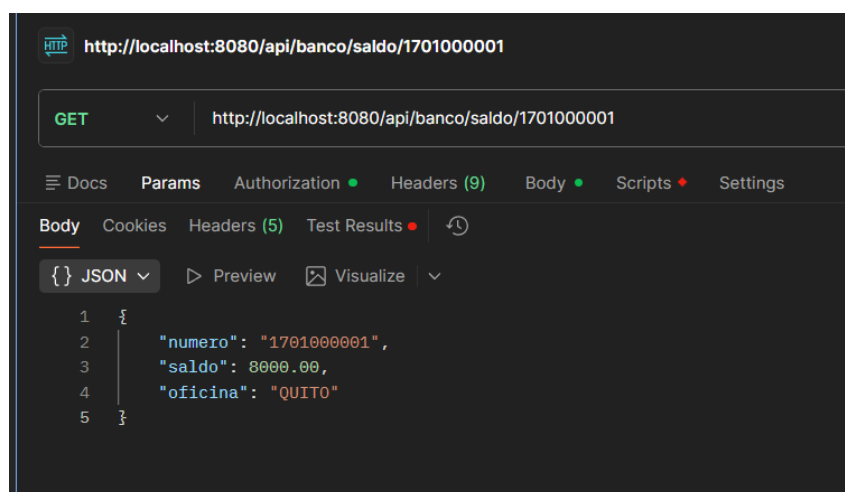


Figura 6: Respuesta JSON de Quito usando Postman.

### 3.3 Consulta distribuida de clientes

Finalmente se ejecutó una consulta para obtener el listado completo de clientes registrados en ambas oficinas.

El servicio realizó dos consultas independientes, una hacia la base de datos de Cuenca y otra hacia la base de datos de Quito. Posteriormente, los resultados fueron combinados y enviados como una única respuesta al usuario.

Esta prueba demuestra el funcionamiento de una consulta distribuida, ya que la información proviene de diferentes nodos físicos, pero es presentada de forma transparente como un único

conjunto de datos.

Endpoint utilizado: GET /api/banco/clientes

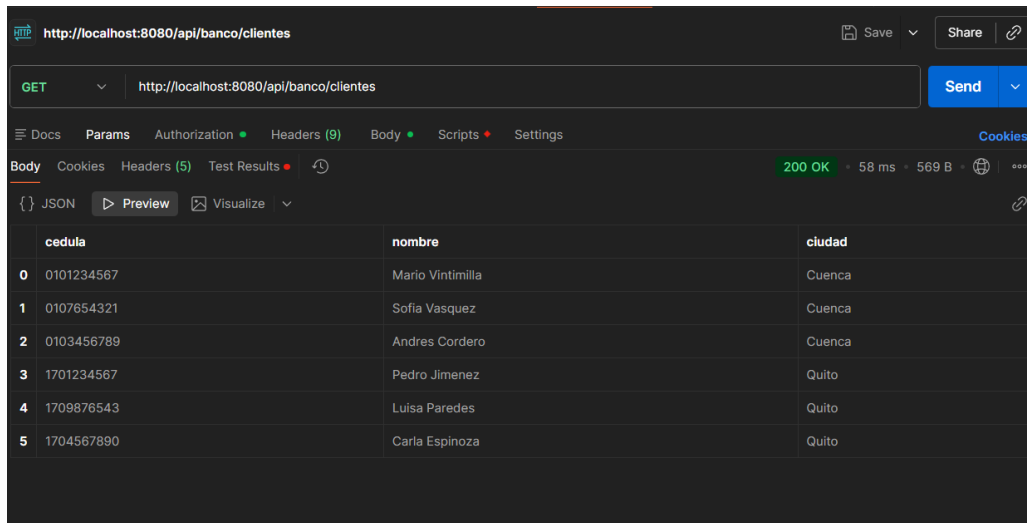


Figura 7: Respuesta JSON con los seis clientes con Postman.

## 4 Conclusiones

1. La implementación permitió comprobar el funcionamiento de una base de datos distribuida mediante el uso de dos nodos PostgreSQL independientes administrados con Docker.
2. El uso de múltiples DataSource en Spring Boot facilitó la conexión simultánea a diferentes bases de datos, permitiendo seleccionar automáticamente el nodo correspondiente según el prefijo del número de cuenta.
3. Las pruebas realizadas demostraron que las consultas fueron ejecutadas correctamente tanto para el nodo de Cuenca como para el de Quito, además de validar la unión de información proveniente de ambos servidores.
4. La práctica permitió comprender la importancia de las bases de datos distribuidas para mejorar la organización de la información y facilitar el acceso transparente a datos almacenados en diferentes ubicaciones.

## 5 Bibliografía

- Docker Inc. (2025). *Docker Documentation*. <https://docs.docker.com/>
- PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- Spring. (2025). *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Özsu, M. T., & Valduriez, P. (2020). *Principles of Distributed Database Systems* (4th ed.). Springer.